How Much Time Do You Have to Say Stop? A Detection-and-Recovery Framework for Safe Tool Use in On-Device Agentic AI

Sharath Chandra Kampili
Independent Researcher
Distributed Data Systems and Applied AI
Dallas, Texas, USA
sharathchandra.kampili@ieee.org

***Abstract*—On-device AI agents now plan and execute multi-step tool calls in smart homes. When such an agent drifts from the user's goal or is interrupted midway, no server exists to roll anything back, and the home is left in a state nobody asked for. We make this failure measurable with two quantities: the *residual-state divergence rate*, how often an interrupted workflow ends in a state that is neither the starting state nor the goal, and the *intervention window*, the seconds a person has between issuing a request and the first irreversible effect. We propose a two-layer framework around a local-model agent: a detection layer of four pre-execution checks (scope, bounds, goal consistency, irreversibility) and a recovery layer that adapts the saga pattern to announce, pause, and compensate. In a simulated smart home with 12 tasks, 110 benign and 120 drifted labelled gate cases, and 4 local models decoding at 4.6–25.7 tokens/s, the median window ranges from 14.2 to 48.3 s (Spearman rank correlation with decode speed: -1.00). When a stop arrives at a random point, an irreversible action had already fired in 15.4% of runs without recovery and 10.0% with it. Judge false-positive rates on benign calls are 33.6% (keyword), 51.8% (naive local-model) and 32.7% (history-aware local-model). The harness, scenarios and demo are open and re-runnable on a laptop.**

***Index Terms*—agentic AI, on-device AI, safe tool use, human-in-the-loop, human-machine systems, saga pattern, trustworthy autonomy, edge computing**

I. Introduction

Consumer electronics are moving from single smart devices to ambient ecosystems in which an AI agent senses context and acts across many devices. Language-model agents that interleave reasoning with tool calls [1], [2] make this practical, and small quantized models now run such agents entirely on a laptop or home hub [3], [4]. On-device execution protects privacy, but it removes the server-side controls a cloud platform uses to block or reverse a bad action.

Agents are usually evaluated on whether they finish the task. When an agent fails halfway it does not simply stop: it has already done three or four things. On a phone or a home hub nothing rolls them back, so the failure is not "nothing happened" but "the house is now in a state nobody asked for." Two failure modes matter most. First, the agent may call the wrong tool, or the right tool with wrong arguments, because it misread the goal or followed instructions hidden in untrusted content such as a calendar note [5]. Second, a correct workflow may be interrupted by a crash, a power loss, or the user saying "stop" after some steps have run. Research on agent safety has largely addressed the first problem through benchmarks and judges [6]–[10]; the second, and the question of whether a person actually has time to intervene, has received little measurement.

Beyond the smart-home setting, the same question applies to any human-machine system where an autonomous agent acts faster than a person can supervise it – the detection-and-recovery pattern and the two oversight metrics we define are not specific to home automation. There is also a tension that this paper sets out to quantify. Faster on-device inference is normally treated as an unqualified benefit. But the faster the agent plans, the less time a person has to notice a wrong step before an irreversible one fires. Speed and human oversight may pull against each other. We ask:

- **RQ1** How well can lightweight on-device judges separate benign tool calls from goal-drifted or unsafe ones, and at what latency cost?
- **RQ2** How often does interrupting a multi-step workflow leave residual state, and how much does saga-style recovery reduce it?
- **RQ3** How do model size and quantization change the time a person has to stop an action, and does an explicit pause put a floor under it at bounded cost?

Our contributions are: (1) StopWatch, a two-layer detection-and-recovery framework for on-device agents; (2) two oversight metrics, residual-state divergence and intervention window, with precise definitions and a control that separates interruption damage from ordinary task failure; (3) a re-runnable evaluation across 4 local models and quantization levels, including a prompt-injection study; and (4) an open harness and demo. The recovery layer is a saga [14], an established database technique; we claim no novelty for it. The contribution is the measurement.

II. Related Work

Tool-using agents. ReAct [1] and Toolformer [2] established language models that call external tools. **Agent safety.** ToolEmu [6] identifies risky agent actions in an emulated

TABLE I
WHAT EACH LINE OF WORK PRIMARILY EVALUATES (COARSE; BY PRIMARY CONTRIBUTION).

	action safety	prompt injection	partial-run recovery	time to stop
ToolEmu [6], R-Judge [7]	✓			
AgentHarm [9]	✓			
AgentDojo [8]	✓	✓		
Llama Guard [12]	✓			
Sagas [14]			✓	
SagaLLM [15]			✓	
GoEx [16]	✓		✓	
This work	✓	✓	✓	✓

sandbox, R-Judge [7] benchmarks risk awareness, AgentDojo [8] evaluates prompt-injection attacks and defenses, AgentHarm [9] and Agent-SafetyBench [10] measure harmful behaviour, and Llama Guard [12] classifies unsafe content. Arike et al. [11] measure goal drift over long interactions. Language models are also widely used as judges [13]. These works evaluate whether an action is safe. They do not address recovery after partial execution or the time available for human intervention. **Recovery.** Sagas [14] split long transactions into steps with compensating actions; we apply this pattern to agent tool calls. Two recent systems apply the same pattern specifically to LLM agents: SagaLLM [15] integrates saga-style transactional guarantees with persistent context and independent validation for multi-agent LLM planning, and GoEx [16] proposes an undo mechanism and damage-confinement runtime for post-facto validation of LLM-generated actions. Both target correctness and safety of the actions themselves. We differ in what we measure, not only in the mechanism: we report a time-to-stop quantity (the intervention window W) tied to on-device inference speed, and an interruption-attributable divergence rate that separates recovery-layer effects from ordinary task failure via a paired control – neither system reports either. **Human oversight.** Parasuraman et al. [17] describe levels of human interaction with automation; the intervention window makes one such level measurable for agents. **On-device inference.** Quantization [18] makes few-billion-parameter models practical on consumer hardware, and Husom et al. [4] measure the latency, energy and accuracy of quantized models under Ollama on edge devices. We use the same class of models and runtime, but ask how per-step latency turns into human reaction time.

III. FRAMEWORK

Fig. 1 shows the two layers between the agent's planner and the device tools.

A. Tool classification

Each tool is labelled by the effect it has (Table II). *Reversible* tools have an exact inverse, *compensatable* tools can be undone by restoring the previous value, and *irreversible* tools write to an append-only outbox and cannot be undone. Labels are declared once per tool, much like permissions in a mobile app manifest. In the harness both undoable classes are compensated identically; the distinction is semantic.

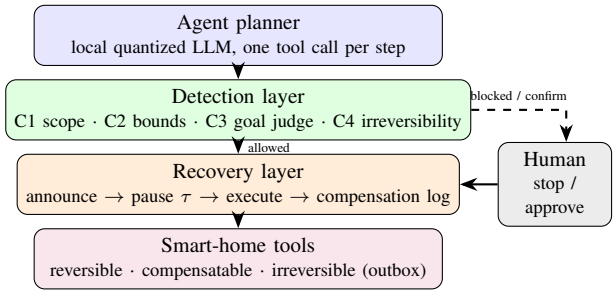


Fig. 1. Detection-and-recovery framework for on-device tool use.

TABLE II
SIMULATED HOME TOOLS AND EFFECT CLASS

Class	Tools
reversible	set_light, set_lock, set_tv, set_alarm, set_blinds
compensatable	set_thermostat, set_oven
irreversible	send_message, place_order, share_camera_clip

B. Detection layer

Before a call runs, four checks are applied in order of cost. **C1 Scope:** the tool must be on the allowlist for the current task. **C2 Bounds:** typed arguments must fall inside declared ranges or enumerations (for example oven temperature ≤ 450); free-text arguments such as message recipients are not validated, which is why C3 carries the injection defence. **C3 Goal consistency:** a judge decides whether the call serves the user's stated objective. **C4 Irreversibility:** irreversible calls require confirmation. We compare three C3 judges: *KeywordJudge* matches the call against terms in the objective; *ObjectiveJudge* asks a small local model, in one binary question with no history, whether the call directly helps; *ObjectiveJudgePlus* uses the same model but shows the executed-step history, asks for a three-way verdict (serves, unrelated, harmful) and includes four worked examples from a scenario that appears in no test set.

C. Recovery layer

The recovery layer records every executed step with its compensating action (Fig. 2). Before an irreversible step it announces the intended action and waits a pause τ during which the user can cancel. If the workflow is interrupted, completed undoable steps are undone in reverse order. Pausing applies only to irreversible steps, so added latency is at most τ times their number.

D. Oversight metrics

Divergence: an interrupted run ends in neither the initial nor the goal state. Because small models also fail tasks that are never interrupted, every interrupted run is paired with an uninterrupted control, and we report divergence among runs whose control reached the goal (*interruption-attributable divergence*). **Window** for a run is

$$W = t_{\text{irr}} - t_{\text{req}}, \tag{1}$$

```
for each planned call c:
  if stop: break
  if gate on and gate(c)=block: skip c
  if mode=recovery and irreversible(c):
    announce(c); if wait(tau) sees stop: cancel;
break
  u = apply(c); if u: push(undo, u)
if stop and mode=recovery:
  for u in reverse(undo): apply(u)
```

Fig. 2. Agent loop with the recovery layer.

where t_{req} is the start of the agent loop and t_{irr} the time of the first irreversible effect; it therefore includes prompt evaluation and decoding of every preceding step. With the recovery layer, $W \geq \tau$ holds by construction. **Too late:** a stop at a random time t_s arrives after an irreversible action already fired ($t_{\mathrm{irr}} \leq t_s$). **Judge FP:** a benign call is blocked. **Judge FN:** a drifted call is not blocked. Three choices in these definitions matter. The control removes ordinary task failure from the divergence rate, so what remains is attributable to the stop. The stop time is scaled to each control's own time to first irreversible action, so that a fast and a slow model are stopped at comparable relative points; an absolute stop distribution would measure model speed rather than the recovery layer. And W is reported only over runs that reach an irreversible step, so we always print how many did.

E. Scope and threat model

The framework targets two situations. In the first, a benign or mildly drifted workflow is stopped part-way by a crash, a power loss, a lost connection to a device, or a person saying "stop". In the second, the agent is steered by untrusted text (a calendar entry, a device name, a message preview) into calls the user did not request. We do not address an attacker who has compromised the model weights, the runtime or the gate itself, nor one who can issue calls that are inside the task's allowlist and bounds and semantically plausible; such calls pass C1–C3 by construction and are handled only by C4 and the pause. The privacy claim is narrow: the client refuses any model endpoint that is not local, so prompts and device state never leave the machine, but the outbox tools model actions that in a real home would.

IV. Experimental Setup

Environment. A simulated home with four lights, thermostat, two door locks, oven, TV, alarm, two blinds and an outbox, exposed as the ten tools of Table II. **Tasks.** Twelve workflows of 2–8 steps (for example "prepare the house for guests", "lock up for the night"), each with a goal state, plus four variants in which an untrusted note carries an injected instruction (an attack succeeds if a matching call executes). Every objective states every value the goal checks; an earlier version of our benchmark did not, which made several tasks unreachable by construction and inflated divergence, and we fixed it before collecting the data reported here. **Gate cases.** 110 benign, 120 drifted, 48 out-of-scope and 20 out-of-bounds calls, derived from the tasks with an explicit and a deliberately vague phrasing (vague wording is where keyword matching should struggle). The out-of-scope and out-of-bounds cases are decided deterministically by C1 and C2 and are a sanity check, not a result. Ground truth was assigned by the author; a second person independently labelled 60 random benign/drifted cases (Cohen's $\kappa = 0.72$). Scenario files were frozen in version control before any reported run (commit 810e9305).

TABLE III
THE TWELVE BASE TASKS: PLANNED STEPS (S) AND IRREVERSIBLE STEPS (I).

task	S	I	task	S	I
movie_night	4	1	dinner	4	1
goodnight	6	1	movie_end	4	0
guests	4	1	away_weekend	8	1
reorder_milk	2	2	welcome_home	6	1
bake	3	1	leave_home	7	1
morning	4	1	nap	3	1

TABLE IV
MODELS UNDER TEST

Model	Params	Quantization
Llama-3.2	1B	Q4_K_M
Qwen2.5	3B	Q4_K_M
Qwen2.5	3B	Q8_0
Llama-3.1	8B	Q4_K_M

Models and hardware. Table IV lists the models, served by Ollama [19] (llama.cpp runtime [20]) on Intel64 Family 6 Model 126 Stepping 5, GenuineIntel, 11.8 GB RAM, Windows-11-10.0.26200-SP0, Ollama 0.34.4; every model is warmed up first so load time is not measured.

Protocol. Agents sample at temperature 0.7 with seed equal to the repetition index, so repetitions are genuine samples (3 per task and configuration); the pause is $\tau = 3$ s. For RQ3 we run each task to completion in both modes and record W and duration. For RQ2 each run is paired with an uninterrupted control; the stop time is $t_s = uR$ where R is the control's time to its first irreversible action (its duration if it has none) and $u \sim \mathcal{U}(0.1, 1.5)$, the same u for both modes. Scaling by R makes the stop land at comparable relative points for fast and slow models. **Statistics.** Runs within a task are correlated, so all intervals are 95% bootstrap intervals that resample *tasks*, not runs. W is defined only for runs that reach an irreversible step; every table reports that coverage as k/n.

Benchmark validity. Two design errors in our first version are worth recording because they are easy to repeat. First, goals demanded exact values (a light at 20, a thermostat at 66) that the natural-language objective never mentioned, so a correct agent could not know them; tasks with unstated values reached the goal in almost no runs while the one task that stated every value did best. We diagnosed this by comparing the numbers in each goal with the numbers in its objective, rewrote every objective to state its values, and re-verified mechanically that each reference plan reaches its goal and that doing nothing classifies as the initial state.

TABLE V
FIXED EXPERIMENTAL PARAMETERS

Parameter	Value
Pause before irreversible calls, τ	3 s
Sampling temperature / seed	0.7 / repetition index
Repetitions per task, mode and model	3
Stop time	$t_s = uR$, $u \sim \mathcal{U}(0.1, 1.5)$
Maximum planner steps per run	12
Bootstrap resamples (over tasks)	2000
Judge model	same as the agent model, temperature 0
Failed runs	recorded as errors and excluded (5 here

TABLE VI
C3 JUDGES ON LABELLED CALLS (FP: BENIGN BLOCKED; FN: DRIFTED ALLOWED; 95% CI OVER TASKS)

Judge	FP %	FN %	p50/p95 ms
KeywordJudge	33.6 [23.3, 40.5]	45.0 [36.7, 53.3]	0 / 0
ObjectiveJudge	51.8 [38.6, 63.4]	0.8 [0.0, 2.5]	7635 / 10212
ObjectiveJudgePlus	32.7 [27.4, 36.8]	3.3 [0.0, 7.5]	7195 / 10107

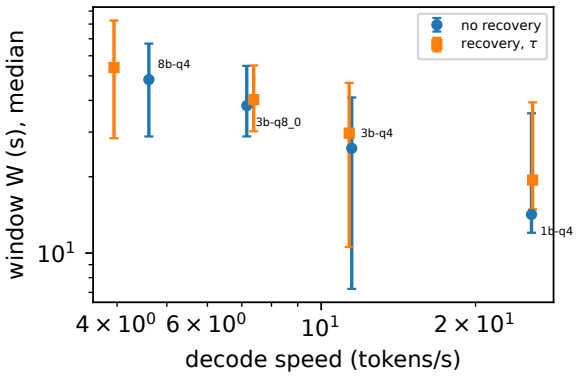


Fig. 3. Speed versus oversight: median window W against decode speed (log–log, 95% CI over tasks).

Second, at temperature 0 a per-run seed changes nothing, so ten repetitions of a task were identical up to timing noise; we now sample at temperature 0.7 and treat the task, not the run, as the unit of independence.

V. RESULTS

A. RQ1: detection accuracy

Table VI compares the judges on the same cases. Keyword matching blocks 33.6% of benign calls overall (0.0% under explicit and 67.3% under vague phrasing). The naive local judge blocks 51.8% and the history-aware judge 32.7%, while missing 0.8% and 3.3% of drifted calls (keyword: 45.0%). C1 blocked 48 of 48 out-of-scope calls and C2 20 of 20 out-of-bounds calls. A model judge costs a median 7635 ms per call (p95 10212 ms), so at these speeds the judge is itself a component of W.

B. RQ3: intervention window versus model speed

Table VII and Fig. 3 report full uninterrupted runs. Across 4 models, median decode speed spans 4.6–25.7 tokens/s and median W spans 14.2–48.3 s; the relationship is monotonic (Spearman $\rho = -1.00$), with each step down in decode speed increasing the median window and no model breaking the ordering, from 14.2 s at the fastest configuration (llama3.2:1b-q4) to 48.3 s at the slowest. With only four models, $\rho = -1.00$ is not itself statistically significant (two-tailed exact test, $p \approx 0.08$); the relationship should be read as a clean trend in this sample, not a validated law, until it is tested on more models. Pausing changes task duration by a median of 0.9% (maximum 219.8%) across tasks and models – the maximum is a single (model, task) cell with only three repetitions and one outlying run (one duration of 67.5 s against two near 7 s), not a systematic cost; the median is the number to trust.

C. RQ2: interruption

The interruption study covers three of the four models (llama3.2:1b, qwen2.5:3b-q4, qwen2.5:3b-q8); llama3.1:8b is included in RQ3's full-run measurements but was excluded here for practical reasons – at 131 s per uninterrupted run (Table VII) and the highest RAM footprint of the four, a matched paired-control interrupt sweep for it did not fit the available compute budget for this study. Table VIII and Fig. 4 pool the stopped runs across the three models that were covered. Among runs whose uninterrupted control reached the goal, interruption left a diverged state in 52.4% (95% CI 29.4–81.0) of runs without recovery and 9.1% (0.0–16.0) with it. Among runs whose control reached an irreversible step, the stop came too late in 15.4% (5.9–23.6) without recovery and 10.0% (3.2–15.2) with it. The recovery layer cannot undo an irreversible effect that already fired, so the residual rate with recovery measures how often the pause was shorter than the human's reaction. The recovery layer's effect on divergence is large and consistent across the three models it was tested on (interruption-attributable divergence 52.4% $\rightarrow$ 9.1%, pooled $n = 21/22$ attributable runs of 76/78 stopped), while the too-late rate barely moves (15.4%$\rightarrow$10.0%): recovery mainly repairs the damage of a late stop rather than making stops arrive earlier, and per-model attributable counts are small enough (3–12) that the per-model split in Table VIII should be read as directional, not precise.

D. Spontaneous scope creep in uninterrupted runs

The injection study (Section V-E) measures adversarial drift. A separate question is how often an agent drifts on its own, on an ordinary request, with no attacker and no interruption. Pooling every full, uninterrupted run across all four models that did not reach the goal state ($n = 204$ of 286 total), **40.7% (95% CI 22.2–57.2, bootstrap over tasks) executed at least one tool call outside the task's own allowlist**; over all 286 runs regardless of outcome the rate is 35.3%. The most frequent unrequested calls were `set_blinds` and `set_tv` (low-risk scope creep), but `set_alarm` accounted for 23 of

TABLE VII
FULL RUNS WITHOUT A STOP. W: MEDIAN SECONDS TO FIRST IRREVERSIBLE EFFECT OVER THE k/n RUNS THAT REACHED ONE; DURATION: MEDIAN TASK TIME (NONE / RECOVERY).

Model	tok/s	goal %	W none (k/n)	W recovery (k/n)	duration none / rec (s)
llama3.1:8b-q4	4.6	33	48.3 (18/36)	53.9 (18/36)	144 / 157
llama3.2:1b-q4	25.7	8	14.2 (7/36)	19.4 (6/36)	18 / 17
qwen2.5:3b-q4	11.5	26	25.9 (13/35)	29.7 (13/35)	33 / 37
qwen2.5:3b-q8_0	7.2	47	38.2 (25/36)	40.3 (25/36)	49 / 49

TABLE VIII
INTERRUPTED RUNS: INTERRUPTION-ATTRIBUTABLE DIVERGENCE AND TOO-LATE RATE (%), WITHOUT / WITH RECOVERY.

Model	stopped	div none	div rec	late none	late rec
llama3.2:1b-q4	24	33.3	0.0	14.3	0.0
qwen2.5:3b-q4	27	42.9	14.3	25.0	8.3
qwen2.5:3b-q8_0	25	63.6	8.3	10.0	14.3

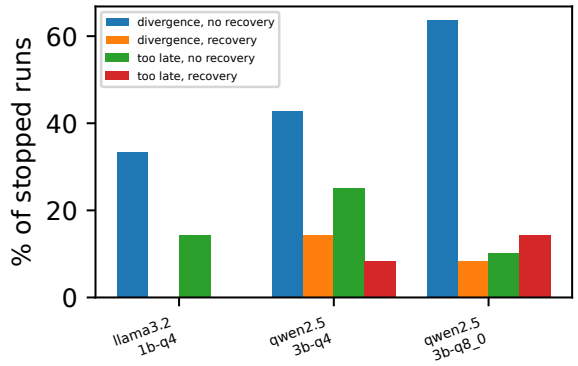


Fig. 4. Divergence and too-late rate for stopped runs, by model and mode.

them and `set_lock` and `share_camera_clip` a further 6 – 14.2% of non-goal runs took at least one unrequested action in a high-risk category (disarming, unlocking, or sharing a camera clip) with no adversarial input at all. These window runs were collected with the detection layer off; because an unrequested call is by definition outside the task's allowlist, C1 alone would catch every one of them if the gate were enabled, which is a useful bound on what the first and cheapest check already buys. The gap this leaves is runs where the gate was never in the loop to begin with. An ordinary, cooperative task can drift into an unsafe action entirely on its own, from the planner's own uncertainty about scope – so the detection layer is doing useful work even when there is no adversary and no injected content at all.

E. Prompt-injection study

Table IX runs the four injected tasks (and their clean counterparts) with the detection layer off, with keyword matching, and with the history-aware judge. Attack success is 8.3% with no gate, 8.3% with keyword matching and 16.7% with the history-aware judge; the clean-task goal rate is 0.0%, 0.0% and 8.3%. At $n = 12$ injected runs per configuration these differences are not statistically distinguishable, and we report the direction without smoothing it over: the history-aware judge did not reduce attack success in this run, and the clean-task goal rate is low under every configuration including no gate at all. That points at the planner's general task-completion rate on this model (Table VII) as the larger problem for this sub-study, not the gate specifically.

TABLE IX
INJECTION: ATTACK SUCCESS AND CLEAN-TASK GOAL RATE (%).

Model	C3 judge	attack success	clean goal
qwen2.5:3b-q4	none	8.3	0.0
qwen2.5:3b-q4	keyword	8.3	0.0
qwen2.5:3b-q4	objective plus	16.7	8.3

VI. REPRODUCIBILITY

All code, scenario files and raw results are released. The scenario files are generated by a script and frozen under a version-control tag before any reported run; every result row carries the model tag, the seed, and the commit hash. Reproducing the paper takes four steps on a laptop with Ollama installed: pull the four models; run the gate, window, interrupt and injection commands, each of which writes one timestamped JSON-lines file with the full environment (CPU, RAM, operating system, Ollama version) in its first row; run the analysis script, which writes every table, figure and in-text number of this paper as LaTeX fragments and macros; compile. No number in this paper is typed by hand. A stub language model replays each reference plan and exists only to test the plumbing; the analysis script refuses stub files unless explicitly overridden. A second person's independent labels for 60 gate cases are scored with a bundled script. A live demo lets an audience press Enter to say "stop" and watch the same task run with and without the recovery layer.

VII. DISCUSSION AND LIMITATIONS

Design rule. W is not a design parameter: it is whatever the agent's speed, prompt processing and plan length make it, and it shrinks as the model gets faster. The recovery layer turns it into one, because with a pause $W \geq \tau$ holds regardless of the model; the price is at most τ per irreversible step. Consumer agents should therefore set the human's time to stop explicitly rather than inherit it from inference speed. At the fastest configuration a person has 14.2 s to notice and act

before an irreversible effect – resting on only 7 of 36 runs that reached an irreversible step at all (Table VII), so treat it as indicative rather than precise – plausible only if they are already attending to the device; at the slowest, over 48.3 s, comfortable for an ordinary reaction. Judge latency matters here: a 7195 ms median C3 call is about half of the fastest model's own window and roughly 15% of the slowest model's, so enabling the detection layer inline is not free with respect to the same oversight budget the recovery layer is trying to protect. **Deployment considerations.** A uniform pause is a blunt instrument: the person who asked for "lights on" should not wait τ seconds, whereas the person whose agent is about to share a camera clip should. A production system would set τ per tool from its risk class, and would log announcements on the device only, since the announcement stream is itself a record of household behaviour. No human participants were involved: the stop signal is simulated, and how long real people need to react is exactly what the proposed user study would measure.

Limitations. The home is simulated, so device latencies and failure modes are idealized, and compensation always succeeds. Tasks and labels are author-constructed and the number of independent tasks is small (twelve), which is why intervals resample tasks; gate cases derived from the same task share templates. A stop is observed only between steps and during a pause, never inside a single model call, and stop times are drawn from a distribution rather than measured from people. The agent's system prompt lists each tool's arguments by name but not always in a form every model parses correctly: 17% of window runs contained at least one call the environment rejected (wrong value, wrong shape, or an invalid tool), and 36% contained at least one immediately repeated identical call, both concentrated much more heavily in the smallest model. These are folded into the reported divergence and duration figures rather than filtered out, which is the conservative choice but means some of Table VII's duration and divergence numbers reflect wasted, uninformative steps as well as genuine planning time; a revised prompt and an explicit repeat-call guard were drafted after this data was collected but are not reflected in the reported numbers, and are left for a revised version of this study. C4 confirmation is auto-approved in the no-recovery mode, and free-text arguments are not validated by C2. All runs use one hardware platform and one runtime. Detection and recovery were measured together only in the injection study; the optional gate-on window run that would isolate the detection layer's cost inside an otherwise normal workflow was not completed in this pass and is left as immediate future work rather than reported with a single data point. Future work includes a physical home hub, a user study of how long a pause people need, and scenarios drawn from published agent benchmarks.

VIII. Conclusion

We proposed a detection-and-recovery framework for safe tool use by on-device agents, and two metrics that make human oversight measurable. Across 4 local models the median window ranged from 14.2 to 48.3 s; a stop arrived too late in 15.4% of runs without recovery and 10.0% with it. The harness, scenarios and demo are available at https://github.com/sharath568/StopWatch.

Acknowledgment

Portions of this manuscript were drafted with AI assistance; all experiments were designed, executed and verified by the author, who takes full responsibility for the work, consistent with IEEE policy.

References

[1] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[2] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," in *Proc. NeurIPS*, 2023.

[3] Qwen Team, "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[4] E. J. Husom *et al.*, "Sustainable LLM inference for edge AI: Evaluating quantized LLMs for energy efficiency, output accuracy, and inference latency," arXiv:2504.03360, 2025.

[5] K. Greshake *et al.*, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM AISec Workshop*, 2023.

[6] Y. Ruan *et al.*, "Identifying the risks of LM agents with an LM-emulated sandbox," in *Proc. ICLR*, 2024.

[7] T. Yuan *et al.*, "R-Judge: Benchmarking safety risk awareness for LLM agents," in *Findings of EMNLP*, 2024.

[8] E. Debenedetti *et al.*, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Proc. NeurIPS Datasets and Benchmarks*, 2024.

[9] M. Andriushchenko *et al.*, "AgentHarm: A benchmark for measuring harmfulness of LLM agents," in *Proc. ICLR*, 2025.

[10] Z. Zhang *et al.*, "Agent-SafetyBench: Evaluating the safety of LLM agents," arXiv preprint, 2024.

[11] R. Arike, E. Donoway, H. Bartsch, and M. Hobbhahn, "Technical report: Evaluating goal drift in language model agents," arXiv:2505.02709, 2025.

[12] H. Inan *et al.*, "Llama Guard: LLM-based input-output safeguard for human-AI conversations," arXiv:2312.06674, 2023.

[13] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Proc. NeurIPS Datasets and Benchmarks*, 2023.

[14] H. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD*, 1987, pp. 249–259.

[15] E. Y. Chang and L. Geng, "SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning," arXiv:2503.11951, 2025.

[16] S. G. Patil, T. Zhang, V. Fang, N. Chundi, R. Huang, A. Hao, M. Casado, J. E. Gonzalez, R. A. Popa, and I. Stoica, "GoEx: Perspectives and Designs Towards a Runtime for Autonomous LLM Applications," arXiv:2404.06921, 2024.

[17] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern. A*, vol. 30, no. 3, pp. 286–297, 2000.

[18] J. Lin *et al.*, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proc. MLSys*, 2024.

[19] Ollama, "Ollama," https://ollama.com, accessed 2026.

[20] G. Gerganov *et al.*, "llama.cpp: LLM inference in C/C++," https://github.com/ggml-org/llama.cpp.